

Lesson 12 — Capstone: A Platformer

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

The six parts of a jump

```
// 1. the impulse
if (buffer > 0 && coyote > 0) {
  vy = -JUMP_SPEED;
  buffer = 0; coyote = 0;      // BOTH, or
  you get a double jump
}

// 2. variable height — cut, do not stop
if (!jumpHeld && vy < 0) vy *= 0.45;

// 3. coyote time — ground remembers itself
if (onGround) coyote = 0.10;
else          coyote -= dt;

// 4. jump buffer — the press remembers
itself
if (jumpPressed) buffer = 0.12;
else          buffer -= dt;

// 5. apex hang
const g = Math.abs(vy) < APEX
      ? GRAVITY * 0.55 : GRAVITY;

// 6. fast fall
const g = vy < 0 ? GRAVITY_UP :
GRAVITY_DOWN;
```

Coyote time and jump buffering are **the same idea in opposite directions**: remember an input briefly. The jump fires when the two windows overlap.

Tune in design units

```
const JUMP_HEIGHT = TILE * 3;      // what
you care about
const TIME_TO_APEX = 0.33;

GRAVITY_UP = 2 * JUMP_HEIGHT / TIME_TO_APEX2;
JUMP_SPEED = GRAVITY_UP * TIME_TO_APEX;
```

```
height = v2 / (2g)      v = √(2 · g · height)
time up = v / g
```

One-way platforms

Needs **direction of travel** and the **previous position**:

```
const wasAbove = previousBottom <= topEdge +
1;
if (!(movingDown && wasAbove)) continue;
// not solid
```

Moving platforms — order matters

1. move the platforms
2. carry any passenger by the same amount
3. move the player, then collide

`ridingPlatform` is set by the **downward** collision test — the same one that sets `onGround`.

Playtesting

Watch. Say nothing.

- "Here, have a go." No instructions.
- Write down every moment of confusion, as it happens.
- Afterwards ask what they were *trying* to do. Never ask if they liked it.

Then **measure**: where did they die?

Done

title · pause · winnable (by somebody else) · survives
blocked storage, resize, hidden tab · controls
discoverable · **one** polished level

Section A — Recall

A1. Name the six parts of a good jump, and for each one name the *moment* it fixes. [6 marks]

A2. Explain how coyote time and jump buffering are the same idea. What single condition fires the jump once you have both? [3 marks]

A3. Why must `coyote` be reset to 0 when a jump fires? [2 marks]

A4. Why multiply `vy` by 0.45 on release rather than setting it to 0? [2 marks]

A5. Why tune the jump in *height* and *time to apex* rather than in speed and gravity? [3 marks]

A6. What two pieces of information does a one-way platform test need that an ordinary solid test does not, and what goes wrong without the second one? [3 marks]

A7. Give the three rules of watching somebody playtest your game. [3 marks]

Section B — Predict the output

B1. You want a jump 96 px high taking 0.3 s to the top. Work out `GRAVITY_UP` and `JUMP_SPEED`. [4 marks]

B2. `JUMP_SPEED` is 600 and gravity is 1800. How high is the jump, and how long to the apex? [3 marks]

B3. Coyote time is 0.1 s and the buffer is 0.12 s. The player walks off a ledge at $t = 0$ and presses jump at $t = 0.08$. Does it jump? Now they press at $t = 0.14$. Does it? Explain both. [4 marks]

B4. The player presses jump at $t = 0$ while falling and lands at $t = 0.09$. The buffer is 0.12 s. What happens, and what would happen with no buffer? [3 marks]

B5. A platform moves up at 60 px/s and `dt` is 0.016. The player stands on it, and the player is moved *before* the platform. Describe what happens over a second. [3 marks]

Section C — Find and fix the bug

C1. The player can jump twice in mid-air, every time. [3 marks]

```
if (buffer > 0 && coyote > 0) {  
    player.vy = -JUMP_SPEED;  
    buffer = 0;  
}
```

C2. Holding the jump key makes the player bounce repeatedly the moment they land. [4 marks]

```
if (keys[" "]) { buffer = JUMP_BUFFER; }  
if (buffer > 0 && coyote > 0) { player.vy = -JUMP_SPEED; coyote = 0; }
```

C3. Releasing the jump key feels like hitting an invisible ceiling. [3 marks]

```
if (!jumpHeld && player.vy < 0) { player.vy = 0; }
```

C4. The player slowly sinks into a platform that is rising, and is left floating when it falls.

[4 marks]

```
movePlayer(dt);
resolveCollisions();
for (const p of platforms) { p.y += p.vy * dt; }
```

C5. Jumping up through a one-way platform snaps the player onto its top instead of passing through.

[4 marks]

```
if (tile.oneWay && movingDown) { /* land on it */ }
```

Section D — Build it

This is the capstone. You have 60 minutes of build time, and then you are going to watch somebody else play it, so leave time for that.

One level, finished. Not four sketches.

- ☐ **Checkpoint 1.** Start from your lesson 5/6 tilemap game. Get gravity and a basic jump working, tuned from `JUMP_HEIGHT` and `TIME_TO_APEX` rather than from raw numbers.
- ☐ **Checkpoint 2.** Add all five refinements, each on a key so you can switch it off. Draw `coyote`, `buffer`, `vy` and `onGround` on screen.
- ☐ **Checkpoint 3.** Set coyote time to 0.5 s and play. Then find the largest value you cannot notice. Write it down.
- ☐ **Checkpoint 4.** Add one-way platforms and one moving platform, with the platforms moved first.
- ☐ **Checkpoint 5.** Add the rest: camera with a dead zone, one enemy that is beatable, coins, a sound per event, particles on landing and on pickup, a title screen, a pause, and a save of the best time. All of this is lessons 6–11; none of it is new.
- ☐ **Checkpoint 6.** Build **one** level that is winnable in about 60 seconds. Play it five times yourself and fix the worst thing each time.
- ☐ **Checkpoint 7. Playtest.** Swap machines with somebody. Say nothing at all. Write down every moment they hesitate, every wrong assumption, and everywhere they die. One sheet of paper.
- ☐ **Checkpoint 8.** Make exactly **one** change based on what you saw, then have them play again and see whether it helped.

THE HARDEST INSTRUCTION IN THIS COURSE

During checkpoint 7, **do not help**. Not a hint, not a nudge, not

“try pressing up”;. Every time you help, you destroy the information you were there to collect. Sit on your hands and write.

Section E — Design challenge

NO RIGHT ANSWER

Marked on reasoning, and on naming the cost.

E1. Coyote time and jump buffering are both "remember an input briefly". Name two other places in a game where that idea would help. (You met one in lesson 7.)

E2. You tuned the jump in design units. What else in your game would be better expressed that way? Give three examples, with the units you would use.

E3. Design one new mechanic — dash, wall jump, double jump, grapple. Before any code: what problem does it solve for the player, what must the level design do to make it worth having, and what does it break about the level you already built?

E4. The hard one. Somebody died eleven times at the same jump. Five possible causes: the jump is too long, the landing is not visible, the run-up is too short, the controls were never explained, the penalty for failure is too high. How would you tell which? What would you change **first**, and how would you know if it worked?

Stretch goals

1. A wall jump — a wall-touch test, a coyote window for it, and a push away.
2. A dash with a cooldown and a trail.
3. Record and replay a run. If it does not replay exactly, you have learned

something — and the advanced level fixes it.

4. A browser level editor that prints your level back out as text.

5. **Ship it.** Put it on a free static host, send it to five people, and write down the first thing each of them did.